

Diagnóstico Completo de Segurança e Estabilidade

FAST SERVIÇOS — versão 4.0.32

Análise de riscos, vazamento de dados, criptografia, instabilidades e plano de correção

DATA DA ANÁLISE
26/09/2026
APLICAÇÃO
FAST SERVIÇOS (app principal + Cliente + Motorista + Rotas do Dia + Despesas)
REPOSITÓRIO
adaiascearataiana-gif/fastservicosapp
BACKEND
Supabase (Auth + REST) · Google Drive · backend WhatsApp
ESCOPO
Código-fonte, autenticação, banco de dados, armazenamento local e integrações
MÉTODO
Revisão estática de código, inspeção em navegador e análise de políticas de banco

1. Sumário executivo

A revisão encontrou **um risco crítico** que, sozinho, permite que qualquer pessoa na internet leia, altere e apague **toda a base de dados** do aplicativo (clientes, rotas, despesas, RH, motoristas, documentos e fotos). Esse risco não depende de adivinhar senhas nem de invadir servidores: a "chave" necessária está publicada dentro do próprio HTML do app, e as regras de segurança do banco estão abertas para qualquer um. É a prioridade número um e deve ser corrigida antes de qualquer outra coisa.

Além disso, foram identificados pontos de risco alto e médio relacionados a armazenamento de tokens, ausência de cabeçalhos de segurança (CSP), uso de caixas de diálogo nativas para confirmar senha, e fragilidades no fluxo de compartilhamento de comprovantes. Também foi corrigida, nesta versão, uma falha de código que quebrava todo o motor do aplicativo (um erro de JavaScript que impedia a pré-visualização de despesas e a sincronização de funcionarem).

1
CRÍTICO
5
ALTO
6
MÉDIO
4
BAIXO

2. Metodologia

A análise foi feita por inspeção direta do código-fonte do aplicativo (arquivo único `index.html` com aproximadamente 65 mil linhas, além dos mini-apps Cliente, Motorista, Rotas do Dia e Despesas), execução do app em navegador controlado para observar erros de console e comportamento real, e leitura do script SQL de criação do banco embutido no próprio app. Foram avaliados: autenticação e sessões, criptografia de credenciais,

políticas de acesso ao banco (RLS), armazenamento local (localStorage/IndexedDB), integrações externas (Google Drive e backend de WhatsApp), comunicação entre páginas (postMessage) e cabeçalhos de segurança.

3. Achados por severidade

3.1 — Crítico

CRÍTICO Banco de dados aberto ao público (RLS com "true" em todas as tabelas)

Categoria: Vazamento de dados / Controle de acesso · Local: script SQL embutido, linhas 37532-37700

Todas as tabelas do Supabase têm Row Level Security (RLS) *ativada*, mas as políticas liberam **tudo para todos**: leitura, inserção, atualização e exclusão usam `USING (true)` e `WITH CHECK (true)`. Como a chave pública do projeto (`sb_publishable_...`) está embutida no HTML — e HTML é público — qualquer pessoa com um navegador e essa chave consegue acessar a API REST do banco e ler, modificar ou apagar os dados de todas as empresas que usam o app.

```
-- Exemplo real do script do app (trecho):
ALTER TABLE rotas ENABLE ROW LEVEL SECURITY;
CREATE POLICY "rotas_read" ON rotas FOR SELECT USING (true); -- qualquer um lê
CREATE POLICY "rotas_write" ON rotas FOR INSERT WITH CHECK (true); -- qualquer um grava
CREATE POLICY "rotas_upd" ON rotas FOR UPDATE USING (true) WITH CHECK (true); -- qualquer um altera
CREATE POLICY "rotas_del" ON rotas FOR DELETE USING (true); -- qualquer um apaga
-- (o mesmo padrão se repete em despesas, clientes, motoristas, rh,
-- dados_empresa, destinos, sequencias_itens, motorista_fotos, etc.)
```

Impacto: vazamento total de dados pessoais e financeiros (nomes, endereços, telefones, valores, documentos de CNH/CRLV, fotos), possibilidade de fraude, exclusão ou corrupção da base inteira, e engenharia social usando dados reais dos clientes. É o risco mais grave do sistema.

Correção: substituir as políticas abertas por políticas amarradas ao usuário autenticado. Como o app já usa Supabase Auth (e-mail/senha), o banco deve exigir `auth.role() = 'authenticated'` no mínimo — e, idealmente, filtrar por dono/empresa (`auth.uid()`). O script pronto está no anexo `correcao-rls-supabase.sql`. Enquanto isso não é aplicado, considere o banco como público.

3.2 — Alto

ALTO Token de acesso do Google Drive guardado em localStorage

Categoria: Criptografia / Armazenamento de segredos · Local: linhas 58352 e 58378

O token OAuth do Google Drive (que dá acesso aos arquivos de fotos e documentos) é salvo em `localStorage` sob a chave `fast_drive_access_token`. Qualquer script injetado na página (por exemplo, via um componente de terceiros comprometido) pode ler esse token e acessar os arquivos do Drive do usuário.

```
var TOKEN_KEY = 'fast_drive_access_token';
...
localStorage.setItem(TOKEN_KEY, v); // token persistente no navegador
```

Impacto: roubo de acesso aos arquivos do Google Drive (fotos de entregas, documentos de motoristas) caso haja qualquer XSS ou extensão maliciosa.

Correção: manter o token apenas em memória/sessionStorage e nunca em localStorage; reduzir o escopo OAuth ao mínimo necessário (`drive.file` já é usado — manter); e implementar CSP (ver achado 3.3) para dificultar injeção de scripts.

ALTO Ausência de Content-Security-Policy (CSP)

Categoria: Cabeçalhos de segurança · Local: <head> do index.html

O app não define uma política de segurança de conteúdo. Ele carrega scripts e estilos de vários domínios externos (Font Awesome, Google Fonts, gerador de QR Code, Google Identity Services). Sem CSP, qualquer injeção de script tem caminho livre, e o risco do achado anterior (roubo de token) aumenta bastante.

Impacto: amplifica qualquer XSS; permite carregamento de scripts não autorizados.

Correção: publicar uma CSP restritiva (via `<meta http-equiv="Content-Security-Policy">` ou cabeçalho no host) listando apenas os domínios realmente usados em `script-src`, `style-src`, `img-src` e `connect-src`. Exemplo no anexo de recomendações.

ALTO Dados sensíveis armazenados sem criptografia no navegador

Categoria: Criptografia / Privacidade · Local: localStorage e IndexedDB em todo o app

Dados de clientes, valores de rotas e despesas, dados de RH, sessão do Supabase (`fast_supa_session`) e credenciais locais ficam em `localStorage` em texto puro. Em aparelhos compartilhados ou comprometidos, esses dados ficam expostos. A senha em si **não** é guardada em texto (usa PBKDF2 — bom), mas o restante sim.

Impacto: exposição de dados pessoais e financeiros em caso de acesso físico ao aparelho ou de XSS.

Correção: criptografar os campos sensíveis antes de gravar (Web Crypto AES-GCM com chave derivada do login), reduzir o tempo de vida da sessão, e oferecer "sair e limpar dados locais" com facilidade. Manter a senha apenas como hash PBKDF2 (já correto).

ALTO Compartilhamento de comprovantes aceita mensagens sem validar a origem

Categoria: Injeção / Integridade · Local: linha 58976

O app escuta mensagens de janela (`postMessage`) para receber comprovantes compartilhados, mas **não verifica** `ev.origin`. Se o app for aberto dentro de outra página (iframe) ou em um contexto controlado por terceiros, mensagens forjadas podem injetar comprovantes e disparar o fluxo de leitura automática.

```
window.addEventListener('message', function(ev){
  if (!ev || !ev.data || typeof ev.data !== 'object') return; // falta checar ev.origin
  var d = ev.data;
  if (d.type === 'fast-share-comprovante' && d.file) { ... }
});
```

Impacto: injeção de dados falsos no fluxo de despesas; risco maior se o app for embutido por terceiros.

Correção: validar `ev.origin` contra a origem oficial (GitHub Pages) e usar `targetOrigin` específico no `postMessage` de retorno, em vez de `'*'`.

ALTO Confirmação de senha por caixa de diálogo nativa (prompt)

Categoria: Exposição de credenciais · Local: linhas 3212, 3261, 3268, 3304, 36691, 36757

Várias ações (ativar biometria, alterar senha, remover senha) usam o `prompt()` nativo do navegador para pedir a senha. Esse campo **não mascara** os caracteres, aparece em texto claro na tela e pode ficar registrado em histórico/gerenciadores.

Impacto: exposição visual da senha a quem estiver por perto e a gravação indevida do valor.

Correção: trocar todos os `prompt()` de senha por um modal próprio com `<input type="password">`, autocomplete apropriado e limpeza do valor após o uso.

3.3 — Médio

MÉDIO Sessão e chave do Supabase expostas no cliente

Local: linhas 35537-35538, 56685, 62908

A URL e a chave pública do Supabase estão fixas no código e também exibidas em um campo da interface de configuração. Chaves "publishable/anon" são feitas para o cliente, então isso é aceitável **somente** com RLS correta — que hoje não existe (ver achado crítico). A sessão (`fast_supa_session`) também fica em localStorage.

Correção: manter a chave publishable (é pública por natureza), mas **fechar a RLS**; remover a exibição da chave na UI; e considerar renovação curta de sessão.

MÉDIO Backend de WhatsApp externo sem autenticação visível

Local: linha 55240

O app conversa com `https://fast-whatsapp-backend.onrender.com`. Durante os testes, a chamada falhou por CORS e não há sinal de autenticação (token/chave) na requisição. Um backend sem autenticação pode ser abusado por terceiros e vaziar conversas.

Correção: exigir um token de API no backend, restringir CORS à origem oficial e nunca expor credenciais do WhatsApp no cliente.

MÉDIO Erro fatal de JavaScript quebrava o motor do app (corrigido nesta versão)

Local: linhas 37422 e 35945

Havia um erro `Assignment to constant variable` (uma constante reatribuída) e uma variável indefinida (`todasOk`). Esse erro **abortava todo o bloco principal de código**, o que explica a pré-visualização de despesas ter "parado de funcionar" e a instabilidade geral. **Já corrigido** nesta versão.

Correção aplicada: `const` → `let` e retorno corrigido. Recomenda-se manter verificação automática de erros de console a cada publicação.

MÉDIO Uso de innerHTML com dados dinâmicos

Local: amplamente no app; função `escapeHtml` existe (linha 48058)

O app monta HTML com `innerHTML` em muitos pontos. Existe uma função `escapeHtml`, mas nem sempre é usada. Dados vindos de cadastros (nomes de clientes, descrições) podem, em tese, injetar HTML/script.

Correção: padronizar o uso de `escapeHtml` em todos os pontos que inserem dados do usuário, ou usar `textContent` quando possível.

MÉDIO Service Worker e cache podem servir versão antiga

Local: `sw.js`, `manifest.webmanifest`, atualização automática

O app usa Service Worker e cache agressivo. Há mecanismos de auto-atualização, mas um cache desatualizado pode fazer o usuário rodar uma versão antiga (inclusive com falhas já corrigidas).

Correção: versionar o cache com o número da versão, forçar `skipWaiting` / `clients.claim` e validar a versão no boot.

MÉDIO Arquivos ausentes no pacote enviado (404s)

Local: `manifest.webmanifest`, `fast-logistics-2.4.js/css`, `motorista-admin.js/css`, `datavalid-panel.js`, `desktop-guides-keyboard.js`, `table-viewport.js`, `clear-money-on-tap.js`

O ZIP enviado estava incompleto, gerando vários 404 e recursos não carregados. No repositório completo os arquivos existem. Isso afeta integridade/estabilidade, não é falha de segurança.

Correção: publicar sempre a partir do repositório completo (o deploy desta versão já usa o repositório íntegro).

3.4 — Baixo

BAIXO Recuperação de senha depende de serviço externo/allowlist

Local: linhas 58245-58275

O fluxo de recuperação usa o endpoint do Supabase Auth e uma Edge Function de fallback. Se a allowlist de redirecionamento não estiver exata, o link pode falhar. Há também um caminho local que confirma o e-mail cadastrado.

Correção: manter a allowlist exata e documentar o fluxo; considerar confirmação em duas etapas.

BAIXO Sem limite de tentativas no cliente

Local: fluxo de login

Existe bloqueio local por tentativas, mas o controle real de rate limit deve ser do servidor (Supabase). Depender só do cliente é contornável.

Correção: manter o rate limit do Supabase Auth e monitorar tentativas.

BAIXO Exposição de QR Code via serviço externo

Local: api.qrserver.com

QR Codes de instalação são gerados por um serviço de terceiros, enviando a URL do app para esse serviço.

Correção: gerar QR Code localmente (biblioteca embutida) para não depender de terceiros.

BAIXO Monolito de arquivo único dificulta manutenção e auditoria

Local: index.html (~65 mil linhas)

Todo o CSS/JS em um único arquivo dificulta revisão de segurança e favorece regressões (como o erro fatal corrigido).

Correção: no médio prazo, separar em módulos versionados e adicionar verificação automática de sintaxe antes de publicar.

4. O que já foi corrigido nesta versão (4.0.32)

- **Valor ao editar (despesas e rotas):** um valor como `123,45` virava `12.345,00` ao abrir a edição. Corrigida a função de leitura de moeda (`fastParseMoeda`) para tratar corretamente números e o separador de milhar brasileiro.
- **Pré-visualização de despesas:** restaurada a prévia de nome/local/descrição (mostra "Última despesa: local • categoria • forma de pagamento • valor • data").
- **Rotas do dia:** ao digitar o cliente, o app puxa automaticamente os dados da **última rota** daquele cliente.
- **Login do app principal:** removido o pedido de "criar senha". Agora aparece **somente login e senha** (a criação só existe no fluxo de recuperação). Cliente e Motorista já tinham login/senha próprios.
- **Flash de login da Google:** o script do Google Identity Services não é mais carregado ao abrir o app, e a reconexão silenciosa do Google Drive passou a ter limite de frequência (cooldown de 15 min) — elimina o "flash" repetido da tela da Google.
- **Erro fatal de JavaScript:** corrigido o erro que quebrava todo o motor do app.

5. Plano de ação priorizado

#	PRIORIDADE	AÇÃO	ESFORÇO
1	Imediato	Aplicar o script <code>correcao-rls-supabase.sql</code> no Supabase (fechar RLS para usuários autenticados)	Baixo
2	Imediato	Trocar a senha do administrador e revisar acessos após fechar a RLS (assumir que os dados podem ter sido acessados)	Baixo
3	Alto	Publicar Content-Security-Policy restritiva	Médio
4	Alto	Tirar o token do Google Drive do localStorage e reduzir sessão	Médio
5	Alto	Validar origem no postMessage e trocar prompt() de senha por modal	Baixo
6	Médio	Autenticar o backend de WhatsApp e restringir CORS	Médio
7	Médio	Padronizar escapeHtml em todo innerHTML com dados do usuário	Médio
8	Médio	Revisar versionamento de cache do Service Worker	Baixo
9	Baixo	Gerar QR Code localmente; separar o monolito em módulos	Médio

6. Considerações sobre criptografia

O ponto positivo é que a senha do usuário **não** é armazenada em texto puro: o app usa PBKDF2-HMAC-SHA-256 com 600.000 iterações e salt aleatório, o que é uma boa prática. A autenticação real acontece no Supabase Auth (e-mail/senha), com a credencial local servindo de cache offline. Os problemas de criptografia não estão na senha, e sim na proteção dos **dados** e dos **tokens** guardados no navegador (localStorage/IndexedDB), que hoje ficam em texto puro, e na ausência de CSP que os proteja de injeção.

7. Conclusão

O aplicativo tem uma base funcional sólida e já adota boas práticas de senha. O que separa o FAST SERVIÇOS de um nível de segurança adequado é, essencialmente, **uma única correção no banco de dados**: fechar as políticas de acesso (RLS) para que só usuários autenticados possam ler e gravar. Sem isso, todo o resto fica vulnerável. Recomenda-se aplicar essa correção imediatamente, seguir o plano priorizado e adotar verificação automática de erros a cada publicação para evitar regressões como a que foi corrigida nesta versão.

FAST SERVIÇOS — Diagnóstico de Segurança 4.0.32 · Gerado em 26/09/2026 · Documento técnico confidencial.

Anexos: [correcao-rls-supabase.sql](#) (script de correção do banco) · [recomendacoes-csp.txt](#) (exemplo de cabeçalho de segurança).